

PortfolioOS

Interview Preparation Guide

Stock & Portfolio Risk Analyzer

Core stack	React + FastAPI + Python + NumPy/Pandas + yfinance + Gemini API + Recharts
Deployment	Vercel frontend + Render backend
Core analytics	Volatility, Sharpe ratio, historical 95% VaR, Monte Carlo simulation, portfolio weights and correlations

Use this before an interview: memorize the 60-second pitch, understand the request flow, know what each metric means, and be ready to explain why each technology was chosen.

1. Your 60-second project pitch

Say this first

PortfolioOS is a full-stack stock and portfolio risk-analysis application. The React frontend sends analysis requests to a FastAPI backend, which retrieves historical market data through yfinance and calculates quantitative risk metrics such as volatility, Sharpe ratio and historical Value at Risk. I also implemented Monte Carlo simulation to model 10,000 possible price paths over 252 trading days. For portfolios, the system calculates investment-weighted returns, portfolio risk, VaR and pairwise correlations. Finally, I integrated Google Gemini so the application can explain the calculated metrics in plain English rather than asking the AI to perform the financial calculations. The frontend is deployed on Vercel and the backend on Render.

The one-line version:

"A full-stack quantitative finance dashboard that turns market data into stock and portfolio risk insights, with Monte Carlo simulation and AI-generated explanations."

What makes the project technically interesting

- **Full-stack integration:** React client, FastAPI REST API, external market-data API and Gemini API.
- **Quantitative finance:** historical returns, volatility, Sharpe ratio, VaR and portfolio correlations.
- **Simulation:** 10,000 Monte Carlo paths over 252 trading days.
- **AI used correctly:** deterministic Python code computes the numbers; Gemini explains them to the user.
- **Deployment:** separate frontend and backend services.

2. End-to-end workflow

Single-stock workflow

1. User enters a ticker such as **AAPL** or an Indian Yahoo Finance symbol such as **TCS.NS**.
2. React normalizes the ticker and starts the analysis requests.
3. Axios calls the FastAPI endpoints.
4. FastAPI requests market data through yfinance.
5. Python/Numpy/Pandas calculate risk and simulation outputs.
6. The Gemini API receives the calculated metrics and generates a natural-language explanation.
7. FastAPI returns JSON to the React client.
8. React renders metric cards, charts and the AI explanation using Recharts and standard React state.

The five major stock requests

```
/stock/{ticker}
/stock/{ticker}/risk
/stock/{ticker}/montecarlo
/stock/{ticker}/history
/stock/{ticker}/summary
```

The frontend uses Promise.all() so these can be requested in parallel instead of waiting for each request sequentially.

3. Architecture you should be able to draw on a whiteboard

```
User -> React UI -> Axios -> FastAPI
FastAPI -> yfinance / Yahoo Finance -> historical prices
FastAPI -> Numpy / Pandas -> risk metrics + simulation
FastAPI -> Gemini -> AI explanation
FastAPI -> JSON -> React state -> charts / dashboard
```

4. Quantitative finance - know these cold

Returns - Simple period return: $(P_t - P_{(t-1)}) / P_{(t-1)}$. The code derives a return series from historical closing prices.

Volatility - Standard deviation of returns. Higher volatility means the observed return series moved more aggressively. The project uses it as a risk measure.

Sharpe ratio - Risk-adjusted return measure. The implementation uses a 7% annual risk-free rate, converts it to a daily rate, calculates excess returns and annualizes using $\sqrt{252}$. Higher is generally better, but the exact interpretation depends on the assumptions.

Historical VaR - 95% historical VaR is based on the 5th percentile of historical returns. It answers a question like: "What daily loss threshold was exceeded only about 5% of the time in the historical sample?"

Monte Carlo - Uses estimated mean and standard deviation of returns to generate random future return shocks, then compounds them into simulated price paths. The project uses 10,000 simulations across 252 trading days.

Important interview nuance: VaR is not the maximum possible loss. A 95% VaR says something about the boundary of the worst 5% of modeled/historical outcomes; it does not tell you how bad losses can get beyond that threshold.

Why 252?

252 is a common approximation for the number of trading days in a year. It is used here for annualization and the one-year Monte Carlo horizon.

5. Monte Carlo simulation - explain it simply

Think of it as asking the computer: *'If tomorrow's return behaves like a random draw from the historical distribution, what could a one-year path look like?'* Repeat that 10,000 times, and you get a distribution of possible ending prices.

Inputs: current price, historical mean return, historical standard deviation.

Random step: draw a random daily shock from a normal distribution based on those estimates.

Repeat: 252 times per path and 10,000 paths total.

Outputs: expected terminal price plus low/high scenario summaries and simulated VaR thresholds.

A strong answer to "What are the limitations?"

- It assumes the estimated historical distribution is useful for the future.
- Normal-style shocks can underestimate fat tails and jumps.
- It does not model news, regime changes, liquidity shocks or changing volatility explicitly.
- The output is a scenario distribution, not a prediction of what will happen.

6. Portfolio analysis

The user supplies holdings as **ticker + investment amount**. The backend converts amounts into portfolio weights:

```
weight_i = investment_i / total_portfolio_value
```

Then the application obtains historical return series for each holding and forms the weighted portfolio return series:

```
portfolio_return = sum(weight_i * return_i)
```

From that portfolio return series, the app calculates portfolio volatility, Sharpe ratio and 95% VaR. It also computes pairwise correlations between holdings so users can see whether assets tend to move together.

Why correlations matter

Two assets with high positive correlation tend to move in the same direction, so combining them may provide less diversification benefit. Lower or negative correlation can improve diversification, though correlation is not constant through time.

7. API cheat sheet

Endpoint	Method	Purpose
/stock/{ticker}	GET	Current stock / summary data
/stock/{ticker}/risk	GET	Volatility, Sharpe ratio, historical VaR
/stock/{ticker}/montecarlo	GET	10,000-path Monte Carlo results
/stock/{ticker}/history	GET	Historical closing-price series for charts
/stock/{ticker}/summary	GET	AI-oriented stock summary / Gemini explanation
/portfolio/risk	POST	Portfolio weights, risk metrics and correlations

8. Technology stack - what each tool does

Tool	Role in the project
Python	Backend language and quantitative finance logic
FastAPI	REST API layer; request routing, validation and JSON responses
Uvicorn	ASGI server used to run the FastAPI application
yfinance	Retrieves Yahoo Finance market / historical price data
NumPy	Return calculations, statistics, percentiles, random simulation and correlations
Pandas	Tabular time-series manipulation and alignment of market data
Google GenAI / Gemini	Turns already-calculated risk metrics into plain-English explanations
React	Frontend UI and state management
Axios	HTTP client used by React to call the backend
Recharts	Price / simulation charts in the dashboard
python-dotenv	Loads secrets/config such as GEMINI_API_KEY from environment variables
Vercel	Frontend deployment
Render	Backend deployment

9. Interview questions and strong answers

Why FastAPI instead of Flask?

FastAPI gives a clean modern way to build typed REST APIs, has automatic OpenAPI documentation and request validation through Pydantic, and works well for a lightweight Python analytics backend. For this project, it is simple enough without adding unnecessary framework complexity.

Why React?

The app has several interactive states - ticker input, loading states, risk cards, charts, portfolio holdings and errors. React makes those UI updates component-driven and stateful.

Why yfinance?

The project needed historical market data without building a market-data ingestion system from scratch. yfinance provides convenient Python access to Yahoo Finance data, which is sufficient for this prototype.

Why not let Gemini calculate the risk metrics?

Financial calculations should be deterministic and reproducible. The Python backend calculates volatility, Sharpe, VaR and Monte Carlo outputs first; Gemini is only used to explain the resulting numbers. That separation reduces the risk of an AI model inventing or changing a financial result.

What is the difference between historical VaR and Monte Carlo VaR?

Historical VaR is empirical: it comes directly from the observed return distribution. Monte Carlo VaR comes from a simulated distribution generated using assumptions about the return process. Historical is assumption-light but dependent on the sample; Monte Carlo is flexible but assumption-heavy.

What would happen if Yahoo Finance fails?

The backend should return an API error rather than crashing the whole server. The implementation also contains fallback handling for certain stock-information lookups. In production I would add retries, caching, rate-limit handling and potentially a second market-data provider.

Does the project predict stock prices?

No. Monte Carlo produces scenario distributions based on estimated historical behavior. It should be described as risk/scenario analysis, not as a guaranteed prediction model.

What is the biggest limitation?

Historical assumptions. Markets are non-stationary, returns can have fat tails, volatility changes over time and major events can invalidate historical behavior. The next step would be more robust risk models and more explicit assumptions.

How would you improve scalability?

Cache market data, reduce duplicate provider calls, introduce async/background jobs for expensive simulations, add rate limiting, centralize API clients, and separate the analytics layer from HTTP routing.

How would you improve security?

Keep secrets in environment variables, restrict CORS to the deployed frontend, validate and sanitize ticker/portfolio inputs, add rate limiting, and avoid exposing provider/API errors directly to users.

10. Be ready to explain one piece of code

A good example is the frontend analysis call. Explain it conceptually rather than line-by-line:

"When the user submits a ticker, I normalize it and fire the independent stock endpoints in parallel with Promise.all. Each endpoint has one responsibility - summary, risk, Monte Carlo, history and AI explanation. Once all responses arrive, React stores them in state and the UI renders the corresponding sections. This keeps the data-fetching flow straightforward and avoids unnecessary sequential latency."

11. Error handling and production concerns

- **Yahoo rate limits:** public market-data endpoints can throttle repeated requests. Caching and request deduplication are the first upgrades.
- **External API failures:** show user-friendly errors and avoid exposing raw exception details.
- **Slow Monte Carlo:** move simulation work off the request thread as traffic grows.
- **CORS:** production should allow only the known frontend origin instead of broad development settings.
- **Secret handling:** keep GEMINI_API_KEY in environment variables and never commit the secret.

12. Files you should know

frontend/src/App.js - Main React application: UI state, API calls, stock/portfolio views and chart rendering.

frontend/src/App.css - Application styling and layout.

frontend/package.json - Frontend dependencies and scripts.

frontend/public/index.html - HTML shell, document title, favicon and metadata.

backend/main.py - FastAPI app, endpoints, market-data calls and Gemini integration.

backend/finance.py - Financial calculations and Monte Carlo simulation logic.

backend/requirements.txt - Python dependencies.

.gitignore / environment files - Keeps generated files, virtual environments and secrets out of source control.

13. 5-minute revision sheet

- **What is it?** Full-stack stock and portfolio risk analyzer.
- **Frontend?** React + Axios + Recharts.
- **Backend?** Python + FastAPI + Uvicorn.
- **Market data?** yfinance / Yahoo Finance.
- **Risk metrics?** Volatility, Sharpe ratio, historical 95% VaR.
- **Simulation?** 10,000 Monte Carlo paths x 252 trading days.
- **Portfolio?** Weighted returns, portfolio risk, VaR and pairwise correlations.
- **AI?** Gemini explains metrics; Python computes them.
- **Charts?** Recharts.
- **Deployment?** Vercel frontend + Render backend.
- **No database.** Data is retrieved and analyzed dynamically.
- **Key limitation?** Historical assumptions may not represent future regimes.

14. Final 30-second answer if asked "Tell me about a challenge"

One challenge was dealing with external market-data dependencies. Because the application depends on Yahoo Finance, repeated analysis requests can hit availability or rate-limit issues. The lesson is that third-party data access is not the same as a reliable internal service. In a production version I would add caching, retries with backoff, request deduplication and potentially a second data provider. That would make the analytics layer more resilient without changing the core calculations.

15. Final 30-second answer if asked "What would you build next?"

I would add historical backtesting, richer risk models such as expected shortfall, volatility models that adapt over time, authentication and saved portfolios, market-data caching, and a cleaner service layer separating analytics from API routing. I would also add tests around the financial functions because reproducibility is especially important in quantitative applications.

16. Interview rules for this project

- Say **risk analysis** and **scenario analysis**, not "guaranteed stock prediction."
- Say the **backend calculates** and Gemini **explains**.
- Know what 95% VaR actually means.
- Know why 252 days is used.
- Know the difference between historical and Monte Carlo VaR.
- Know at least one limitation of the current model.
- Be able to draw the architecture in under one minute.

Repository: github.com/ganesan98/portfolios

Prepared as a practical interview revision document based on the current project structure and implementation.